U U U I I U I U I I I U U U I U U I U

*ma · sa · ja · sa · ta · ta · ga — śārdūlavikrīḍitamu, the metre of Pothana's opening verse*

# Chandohasam

## Towards semantically coherent Telugu metrical poetry generation with text diffusion

**Progress seminar · 6 October 2026**

Samvaran Kashyap Rallabandi · Mahesh Emani · Radhe Shyam Salopanthula
IIIT Hyderabad

# The narrative in one page

### 1 The problem

A padyaṃ must balance śabdaṃ (sound) and arthaṃ (meaning); prior work solves one at the expense of the other.

### 2 Chandassu

Akṣara weights, gaṇas, yati and prāsa; three families of metres; why Telugu is harder than English, Chinese or Sanskrit.

### 3 The benchmark

12,673 labelled poems from four sources, audited for metre, meaning and duplication.

### 4 The controller

One DAWG for 37 metres; constrained decoding as a puzzle with no dead ends, for autoregressive and diffusion models.

### 5 Results

Form by construction for every model; the lexical cost of subword tokens; our 120M akṣara-level diffusion model.

### 6 Measuring poetry

Five deterministic metrics from Kāvyālaṅkārasaṅgrahamu, the metrics in design, the final architecture, next steps.

# Problem statement

Compose a Telugu padyaṃ that is metrically exact and genuinely meaningful, from a given meaning (bhāvaṃ).

### Must be in metre

- **gaṇa:** the syllable-weight pattern
- **yati:** a sound match inside the line
- **prāsa:** line-initial rhyme across lines
- all three at once, not as aggregates

### Must mean something

- coherent ideas, contextually appropriate
- the verse must express the given bhāvaṃ
- no closed-form check exists for this

### Must be poetic

- **śabdālaṅkāra:** alliteration, repetition
- **arthālaṅkāra:** simile, metaphor, hyperbole, wordplay
- euphony, density and clarity of diction

## Our answer: Chandohasam

**Deterministic controller**

a DAWG over 37 metres, inside the decoding loop

**Curated benchmark**

12,673 labelled poems with meanings

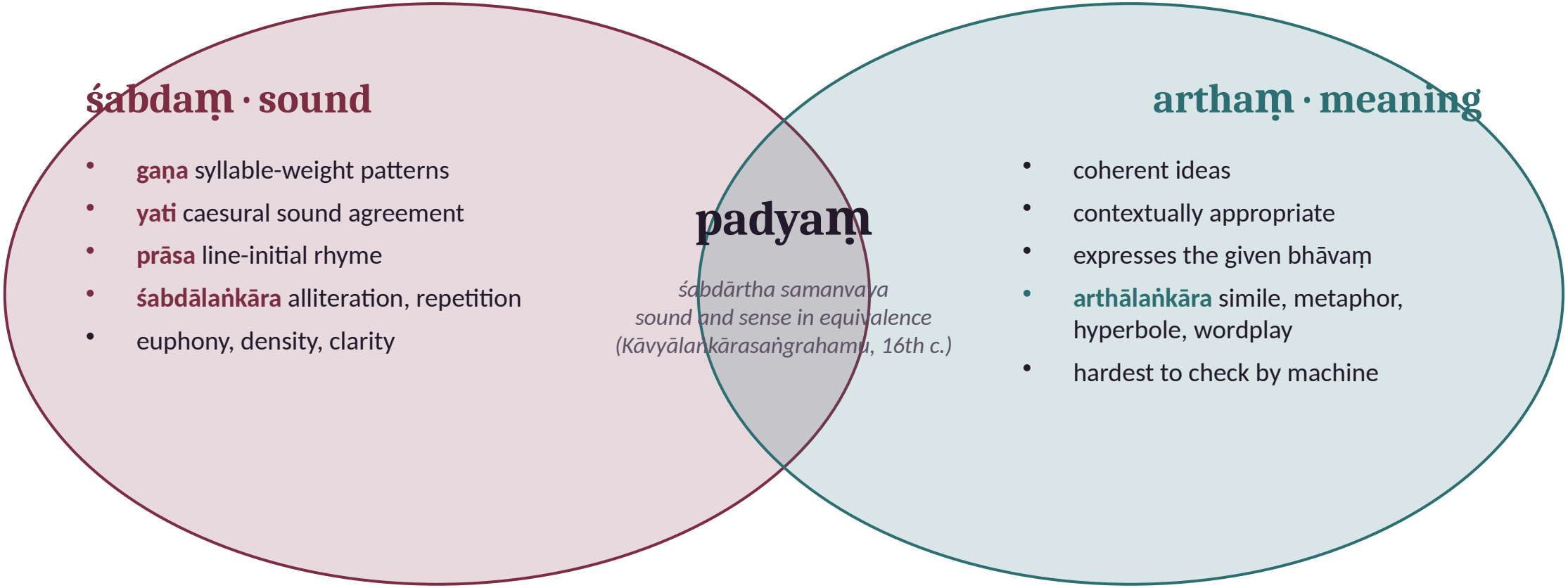
**Evaluation suite**

operationalised from Kāvyālaṅkārasaṅgrahamu

**Coherence metric + RLVR**

from expert judgements; rewards for refinement

# Why poetry is hard: śabdaṃ and arthaṃ in balance




Metre without meaning is noise in perfect rhythm; meaning without metre is prose. Prior work solves one side at the expense of the other.

# Existing methods and their flaws

## Soft route: steer the model, discard failures

- **Deep-speare** (Lau et al., 2018): meter learnt jointly with language, English sonnets
- **SongNet** (Li et al., 2020): rigid formats for Chinese SongCi
- **PoeLM** (Ormazabal et al., 2022): control codes, sample many, keep what passes scansion
- **Bangla** (Amina et al., 2025): rhyme, matra, word-count tokens; follows the structure > 90%
- **Flaw:** constraints are aggregates (a matra total, an end rhyme); semantic drift when several control tokens apply

## Hard route: guarantee form at decode time

- **Constrained decoding** (Ghazvininejad et al., 2016; Lu et al., 2021; Geng et al., 2023)
- **Chandomitra, PINGALA** (Jagadeeshan et al., 2026a, b): near-perfect Anushtubh metre with greedy decoding
- **Flaw:** human raters score coherence at 2.1 to 2.7 on a 5-point scale
- **PoetryDiffusion** (Hu et al., 2023): text diffusion with a learned BERT metrical controller, aims at both

**Metre is close to solved; meaning under metre is not.**
We keep a deterministic metrical controller inside the decoding loop and attack semantic coherence on top of it.

# Why Telugu is different

## 1 Three constraints on every line, at once

- gaṇa grid from the first akṣara
- prāsa: second akṣara of every line repeats a consonant
- yati: a seat inside the line must agree in sound with the line's first akṣara (vaḷi)
- positional rules, not aggregates

## 2 37 metres in three families

- 26 vṛtta: a fixed gaṇa sequence
- 8 jāti: gaṇa classes, lines of varying length
- 3 upajāti: composite stanzas, no prāsa
- one controller must serve all of them

## 3 An alpha-syllabary script

- an akṣara's weight stays pending until the next onset: a following conjunct makes it heavy
- sandhi can hide a yati seat
- subword tokenizers spend 1.2 to 1.4 tokens per akṣara

## 4 An evaluation gap

- metre: a deterministic identifier for Sanskrit, only an approximate checker for Telugu
- meaning: no metric; MAUVE unvalidated on classical poetry
- śabdālaṅkāra identifiers exist only for Sanskrit

# Chandassu and the akṣara: what is annotated

- **akṣara** a syllable; guru (heavy, U) or laghu (light, I)
- **gaṇa** a fixed group of weights: bha UII, ja IUI, sa IIU, na III, ya IUU, ra UIU, ta UUI, ma UUU, ga U, va IU
- **pāda** a line; a slot is a kind of line a metre allows
- **yati** at a yati seat the akṣara must agree in sound (maitri) with the line's first akṣara, the vaḷi
- **prāsa** the second akṣara of every line repeats the consonant of line 1's second akṣara
- **pending weight** a light akṣara before a conjunct is heavy, so a weight is settled only by the next onset

**Annotated: pāda 1 of Pothana's opening verse (śārdūlavikrīḍitamu)**

శ్రీకైవల్య పదంబుఁ జేరుటకునై చింతించెదన్ లోకర

| మ | స | జ | స | త | త | గ |
|---|---|---|---|---|---|---|
| శ్రీ కై వ | ల్య ప దం | బుఁ జే రు | ట కు నై | చిం తిం చె | దన్ లో క | ర |
| U U U | I I U | I U I | I I U | U U I | U U I | I→U |

- **gaṇas** ma sa ja sa ta ta ga: 19 akṣaras; the pāda-final laghu (ర) reads as guru
- **yati 1–13** శ్రీ – చిం: sound-class match with the vaḷi (sarasa yati)
- **prāsa** క in akṣara 2 of all four pādas: శ్రీ·కై, క్ష్మా·కా, ద్రీ·క, నా·కం
- **scansion** the controller settles each weight from the following onset; output of python3 -m chandohasam

# Three families of metres: fixed, class-based, composite

### vṛtta · 26 metres · fixed

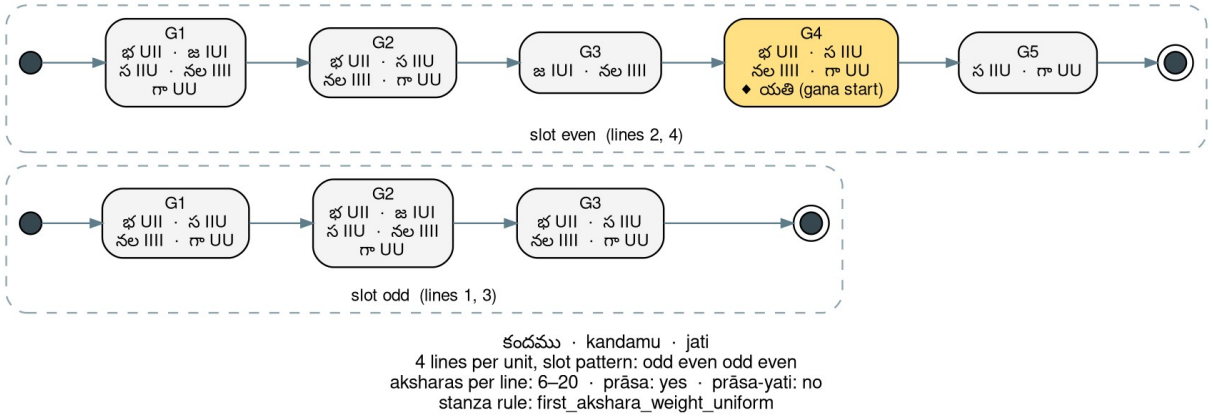
- one gaṇa sequence per pāda, four pādas; prāsa: yes
- **utpalamāla** bha ra na bha bha ra va (20), yati 10
- **śārdūlavikrīḍitamu** ma sa ja sa ta ta ga (19), yati 13
- **sragdhara** ma ra bha na ya ya ya (21), yati 8, 15

### jāti · 8 metres · class-based

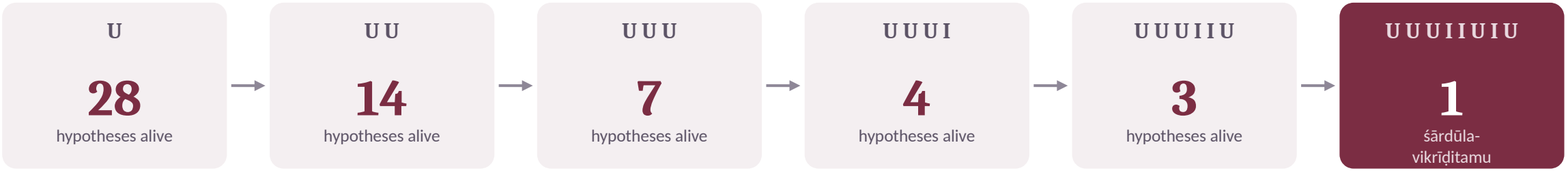
- each position takes any gaṇa of its class, so line length varies; prāsa: yes
- **sūrya** na, ha
- **indra** bha, ra, ta, nala, naga, sala
- **kandamu** odd 3, even 5 kanda gaṇas; yati at gaṇa 4
- yati at the first akṣara of a gaṇa

### upajāti · 3 metres · composite

- **sīsamu** 6 indra + 2 sūrya, then a gīti
- **āṭaveladi** odd 3 sūrya + 2 indra; even 5 sūrya
- **tēṭagīti** 1 sūrya + 2 indra + 2 sūrya
- no prāsa; prāsa-yati accepted in place of yati



# Constrained decoding is a puzzle with no dead ends




*The controller's viable_prefix query on a growing weight prefix: how many (metre, slot) hypotheses can still be completed.*

### 1 The grid acts from the first token

Only tokens whose akṣaras leave at least one completable metre slot are allowed; all others are masked.

### 2 Prāsa and yati join once anchored

The second akṣara of line 1 fixes the prāsa register; each line's first akṣara fixes its yati register.

### 3 Exact liveness: no dead ends

A valid path can never get stuck, so every completed poem satisfies the metre; meaning is left to the model and the evaluation.

# The benchmark: 12,673 labelled poems, four sources

| Rule, labelled verse poems | Bhāgavatamu | Vemana | Kuchimanchi | Chandassu | Total |
|---|---|---|---|---|---|
| **Poems** | 7,366 | 1,164 | 1,611 | 2,532 | **12,673** |
| 1. gaṇa | 99.85% | 87.2% | 96.5% | 85.0% | **95.3%** |
| 2. prāsa, relaxed (strict) | 99.8% (96.7%) | 95.6% (93.4%) | 99.9% (97.7%) | 98.8% (93.7%) | **99.2% (95.9%)** |
| 3. yati, relaxed (strict) | 91.4% (90.7%) | 96.4% (95.7%) | 93.9% (92.9%) | 87.9% (86.6%) | **91.5% (90.6%)** |

*Table 1 of the paper: validity rates measured by the controller; strict values in parentheses.*

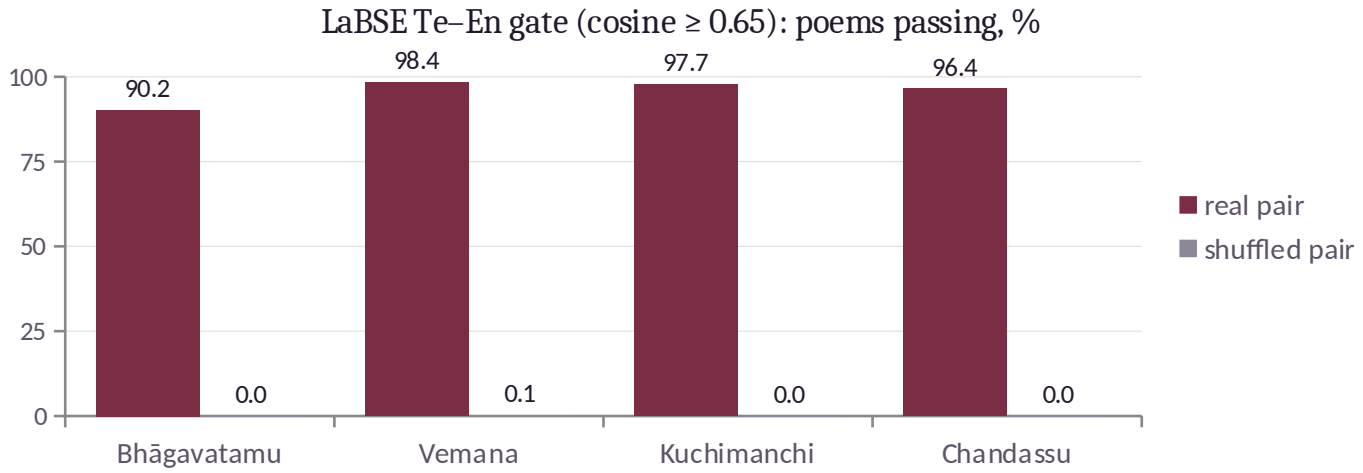
## Sources

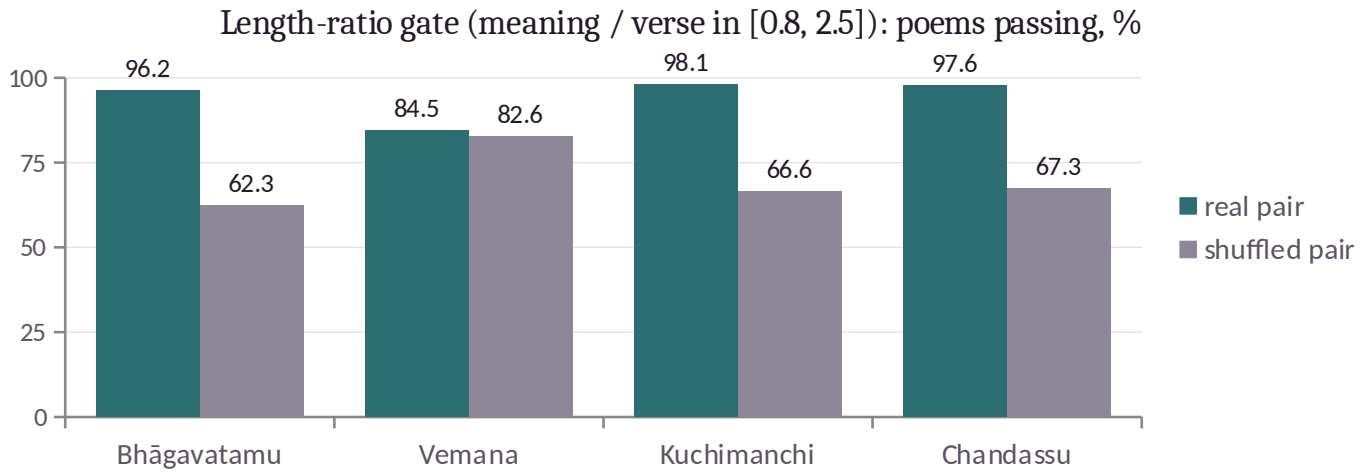
- **Bhāgavatamu** Pothana's Āndhra Mahābhāgavatamu, online compilation with word glosses and meanings: the golden set
- **Vemana** the śatakam
- **Kuchimanchi** Timmakavi's acca-telugu Rāmāyaṇamu; ciravibhava, kukkuṭeśvara and śrībharga śatakams
- **Chandassu** Pavan and Sree (2025), Kaggle release (Boddu, 2025)

## Meanings and labels

- Bhāgavatamu meanings come from the edition; the rest are LLM-generated (Gemini)
- every English meaning is a machine translation of the Telugu bhāvaṃ
- metre labels checked by the controller; gaṇa validity is lowest on Chandassu (85.0%) and Vemana (87.2%)
- quality gauged by metrical correctness, length ratio, embedding similarity, lexical and metrical diversity, author diversity, deduplication

# Data sanity: do the meanings belong to their poems?




## What Table 2 says

- **Fidelity:** poem vs. its Telugu meaning, LaBSE ranks the true pairing first with AUC 0.85–0.94
- **Uniform machine meanings:** the length gate cannot separate Vemana (84.5 vs 82.6)
- **Diversity:** type–token ratio 0.59–0.64 in every file
- **Duplication:** 13 near-duplicate pairs among 12,704 poems (3-gram Jaccard ≥ 0.7)
- **Refrains:** 77.1% of Vemana and 25.3% of Chandassu poems share a pāda; Yule's K 99.9 vs 1.4–4.1, so splits keep refrain groups together
- **Tokens:** Gemma-4 subwords spend 1.23–1.36 tokens per akṣara, motivating a syllable tokenizer

# The DAWG: every legal line of a metre, stored once

**1. Rules as data**

A toy line slot:

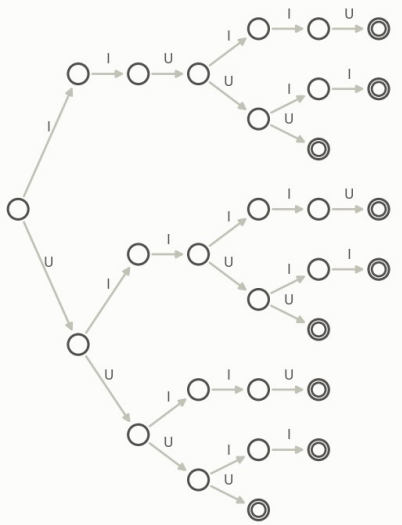
```
Line  →  G G
G  →  UU | IIU | UII
```

Its 9 legal lines:

```
IIUIIU
IIUUII
IIUUU
UIIIIU
UIIUII
UIIUU
UUIIU
UUUII
UUUU
```

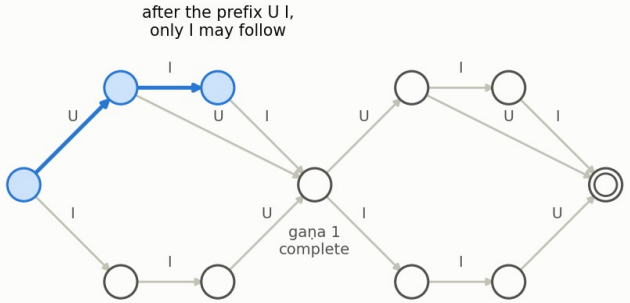
U = guru (heavy)
I = laghu (light)

**2. Trie of every legal line: 29 states**



The branches of gaṇa 2 are stored again for every way to end gaṇa 1.

**3. Minimal DAWG: 11 states, 14 edges, same 9 lines**




Equal futures are merged: the three ways to finish gaṇa 1 lead to one state, and gaṇa 2 is stored once.
A decoder asks the automaton which symbols can follow its prefix (next_symbols) and masks the rest.
The real catalogue: 370,315 legal lines of 37 metres in one labelled DAWG of 1,431 states and 2,514 edges.

Source: toy slot built and minimised in make_figures.py; real counts from writeups/data/dawg_validation.json

## Directed acyclic word graph

A minimal deterministic acyclic automaton over U and I. A line is legal when its weights lead from the start to a final state; lines are bounded, so there are no cycles.

## Merge equal futures

States with the same completions become one, so each shared suffix is stored once: 29 trie states become 11 states and 14 edges for the same 9 lines (Daciuk et al., 2000).

## The decoder’s query

next_symbols: which weights may follow this prefix? After U I, only I. The answer is a lookup, cheap enough for every token of every decoding step.

# One labelled DAWG for all 37 metres

**Rules as data (YAML)**
37 metres, gaṇa classes; 64 prāsa and 92 yati rules from Appakavīyaṃ

↓

**Prosodic grammar per metre**
poem → lines → gaṇa classes → gaṇas → U / I

↓

**Right-linear grammar per line slot**
a vṛtta has one slot; kandamu has two (odd, even)

↓

**Union + subset construction**
a DFA of 1,573 states; accepting states carry (metre, slot) labels

↓

**Label-preserving minimisation**
the DAWG: 1,431 states · 2,514 edges · 370,315 legal lines · < 0.5 s

## Three queries for the decoder

- **identify** walks a poem's scansion lattice; returns every accepting metre and slot, and where each rejected metre became impossible
- **viable_prefix / next_symbols** the symbols that can still lead to a completed metre slot: the valid continuation set
- **segment** the gaṇa divisions and the yati positions

## Validated on the Bhāgavatamu, independently of generation

**99.85%**
7,355 of 7,366 poems identified with their labelled metre

**0**
identified as a wrong metre

**10**
rejected as unclassified

# Inside the decoding loop: the enforcer

## Enforcer state σ, four parts

- **Q** the live states of the metre's automaton the text may be in
- **w** the last akṣaras whose weight is still open (a light akṣara before a conjunct is heavy)
- **prāsa register** the consonant of line 1's second akṣara, which every line's second akṣara must repeat
- **yati register** the line's first akṣara (the vaḷi), with which the akṣara at a yati seat must agree in sound

## For each next akṣara a (or a space or a line break)

1. Settle the open weights in w by the first consonant of a.
2. Move each state in Q along the settled weights; drop those with no move.
3. If a is the second akṣara of a line and breaks the prāsa: DEAD.
4. Drop the states whose gaṇa division puts a on a yati seat where a is not in maitri with the vaḷi.
5. If no state in Q can still reach the end of the poem: DEAD; otherwise return the updated σ.

**A token is allowed when σ stays alive after all of its akṣaras (cached per state).**
Liveness is exact: decoding never reaches a dead end, and every finished poem is valid for the metre, its prāsa and its yati.

# The same controller on autoregressive and diffusion models

### Autoregressive

- **gemma-4-E4B-it, gemma-4-26B-A4B-it**
- the DAWG masks the next-token distribution directly
- initial temperature 0.7
- subword tokens: one token may hold several akṣaras, or part of one

### Block diffusion

- **diffusiongemma-26B-A4B-it-NVFP4**
- proposals committed left to right while allowed and confident
- otherwise decode the first open position
- a backtrack returns committed positions to noise

### Masked diffusion (ours)

- **chandohasam 120M**
- constraint on the positions being resolved; committed context preserved
- 512-token canvas, meaning unmasked
- one token per akṣara; lines decoded left to right within the line

## Three decoding strategies (Algorithm 3)

### Masking only

Remove invalid tokens, renormalise, sample from the tokens that hold 90% of the mass (top-p).

### Masking + backtracking

Checkpoint every third step (keep 15). When stuck (< 3 allowed tokens, < 0.001 of the mass, or 4 steps without a new akṣara) rewind c checkpoints and raise τ by 0.15c, to at most 1.2.

### Hybrid

Walk the allowed tokens by probability (at most 100), collect those that complete the line (stop at 3), and sample among them.

# Results: the constrained-decoding grid

| model | mode | in metre | attested | distinct | single akṣara | strong anuprāsa | obscure | MAUVE |
|---|---|---|---|---|---|---|---|---|
| **Gemma-4 E4B** | free | 0 | 33.7% | 490 | 0.4% | 0.4% | 2.2% | 0.006 |
| | constrained | **555** | 10.7% | 323 | 9.2% | 33.7% | 91.4% | 0.057 |
| **Gemma-4 26B-A4B** | free | 0 | 37.2% | 545 | 2.0% | 9.9% | 0.9% | 0.009 |
| | constrained | **555** | 16.9% | 375 | 16.1% | 48.1% | 91.0% | 0.014 |
| **DiffusionGemma 26B-A4B** | free | 0 | 34.4% | 485 | 4.4% | 19.1% | 30.3% | 0.006 |
| | constrained | **555** | 11.5% | 333 | 28.7% | 71.9% | 98.9% | 0.011 |
| **Chandohasam 120M (ours)** | free | 0 | 17.9% | 183 | 5.9% | 26.7% | 45.9% | 0.006 |
| | constrained | **555** | **36.9%** | **757** | 19.7% | 33.7% | 62.9% | 0.035 |
| **real verse** | – | – | – | – | 5.6% | 2.7% | 10.0% | **0.965** |

*Table 5 of the paper. Grid: 37 metres × 3 topics × 5 seeds = 555 poems per row. In metre = complete, canonical gaṇa, strict prāsa and strict yati, no sandhi assumed. Attested = words of two or more akṣaras found in held-out verse (topic words excluded). MAUVE against real verse of the same metre mix (n = 120 a side, 5 seeds).*

**0 → 555**

poems in metre, free → constrained, for every model

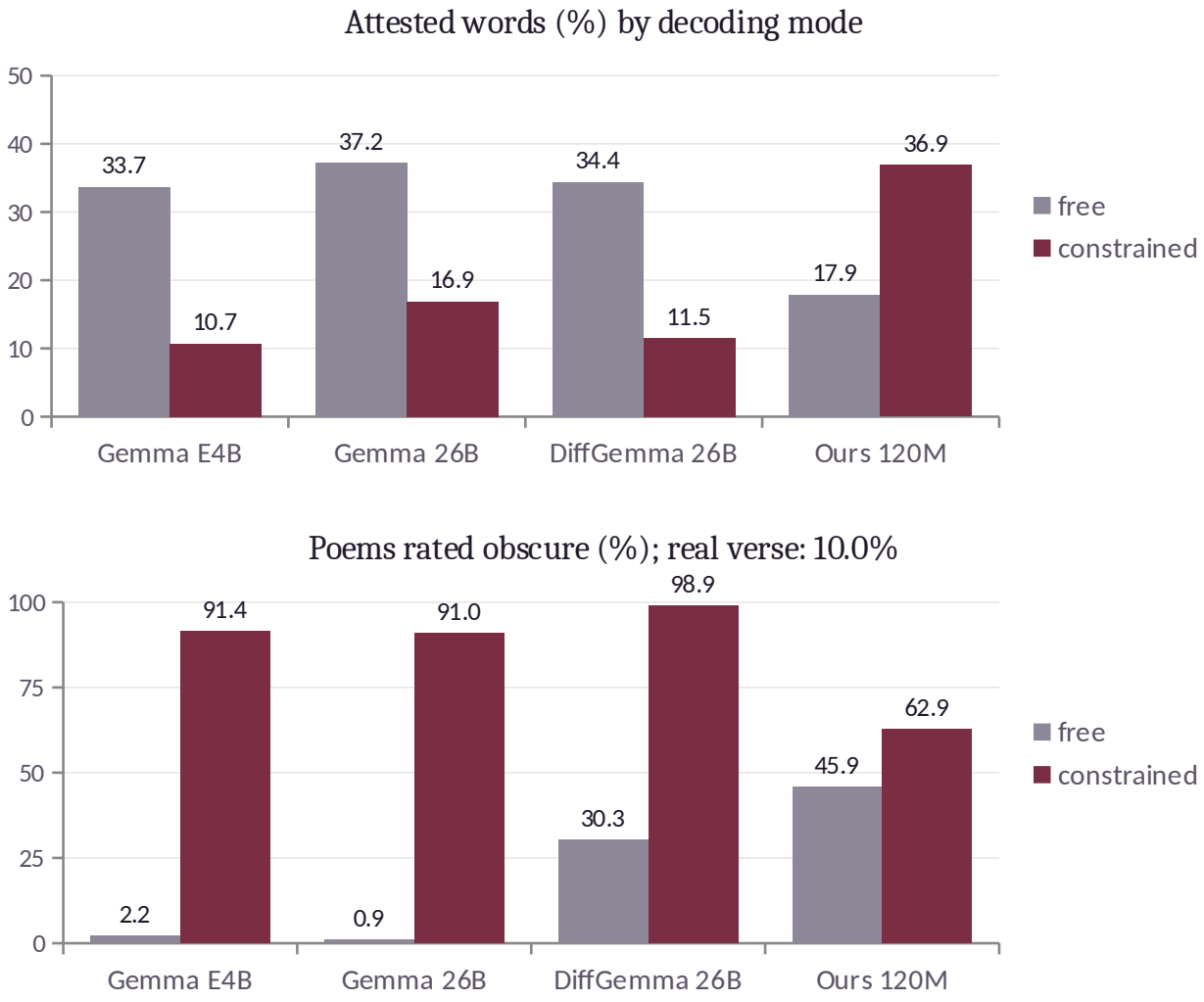
**36.9%**

attested words kept by Chandohasam 120M under constraint; subword models fall to 10.7–16.9%

**0.035 vs 0.965**

best MAUVE of a generated row vs real verse

# Results: form is guaranteed, words are the cost




- **Form by construction.** Free decoding: 0 poems in metre; constrained: 555 of 555 for every model. The decoder guarantees form; nothing is learned.
- **The lexical cost of subword tokens.** Attested words fall from 33.7–37.2% to 10.7–16.9%; strong anuprāsa in 33.7–71.9% of poems (real verse 2.7%); 91.0–98.9% rated obscure (real 10.0%).
- **Akṣara tokens keep the words.** Chandohasam 120M keeps 36.9% attested words and 757 distinct ones, about twice the 323–375 of the subword models. Its deficits reflect pretraining without verse.
- **No one writes like the corpus yet.** MAUVE 0.006–0.057 for every generated row against 0.965 for real verse; the constraint raises it for every model.
- **Scale does not buy coherence.** Layer probes of Gemma E2B and E4B and the recorded token probabilities: all sizes underperform equally.

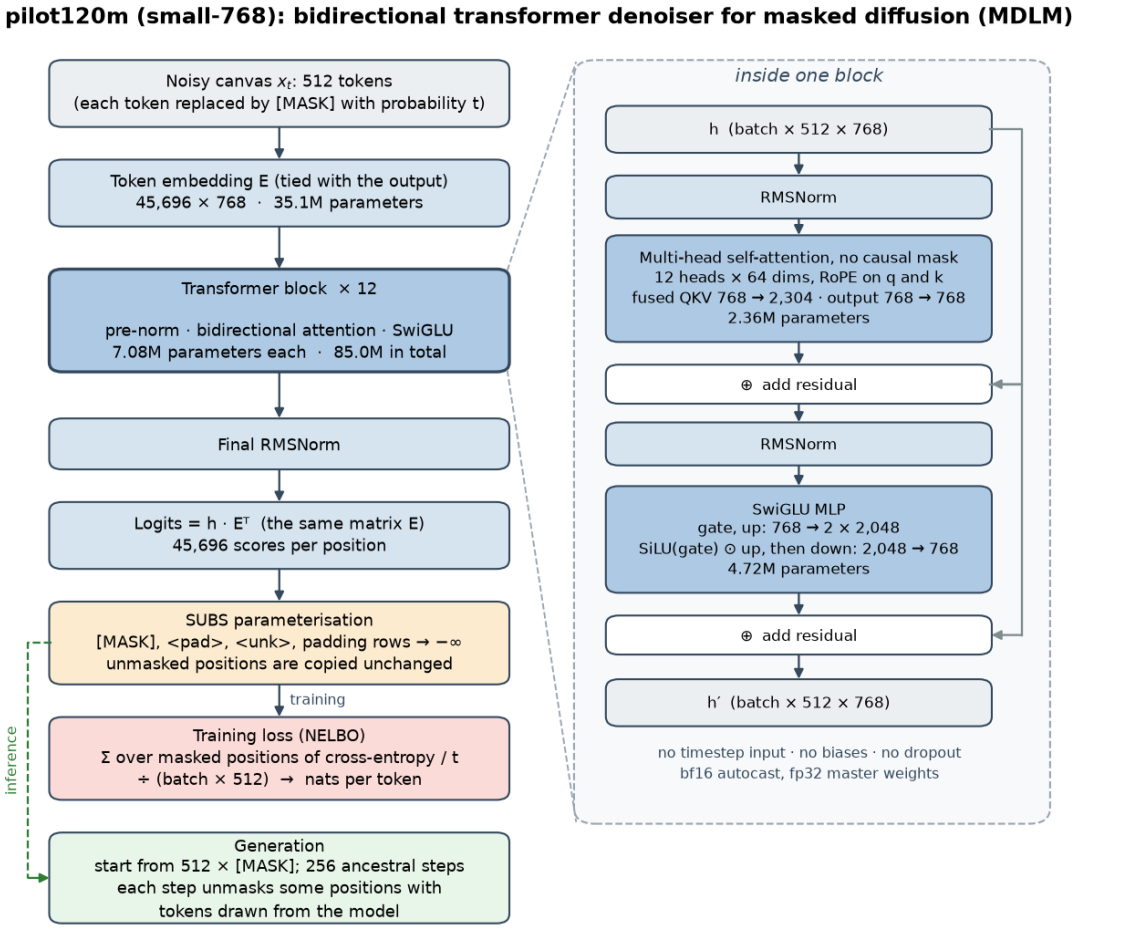
# Pretraining our own diffusion model, one token per akṣara

## Why the tokenizer must be akṣara-based

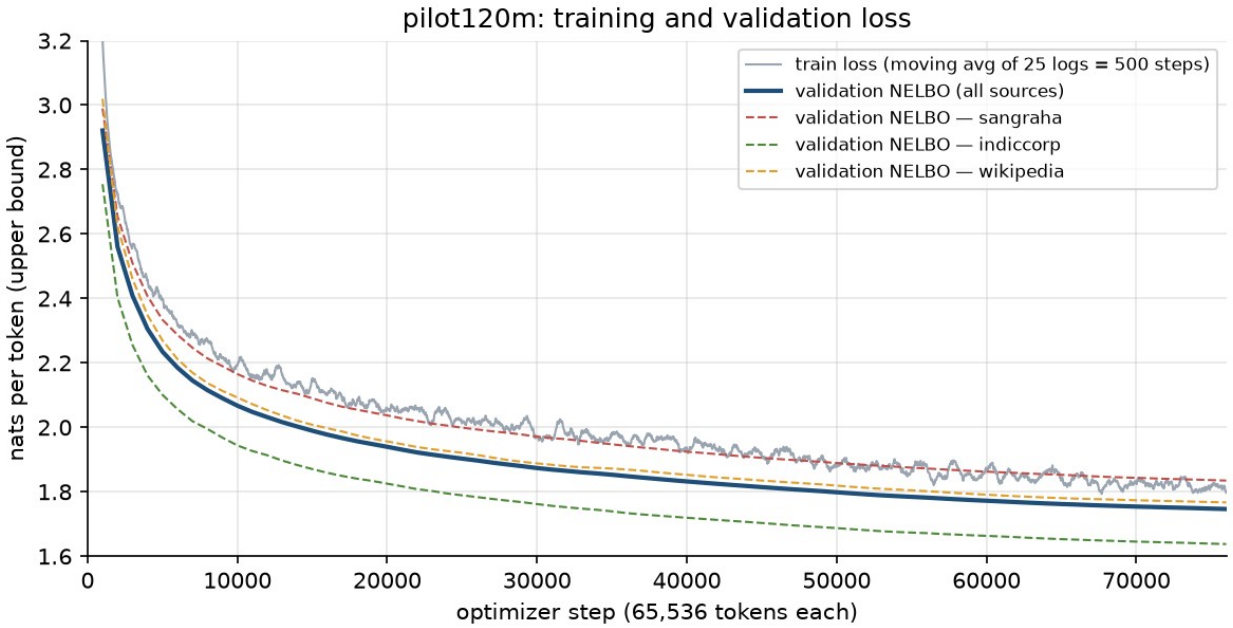
- the constraint and the weights live on akṣaras, settled by the next onset
- subword tokenizers spend 1.2–1.4 tokens per akṣara: a token holds part of an akṣara, or several
- masking at subword granularity strips vocabulary: attested words fall to 10.7–16.9%
- **ours:** 45,591 syllable tokens, akṣara-boundary aware, 99.8% of the akṣara types of our verse

## chandohasam 120M: a masked diffusion language model

- **architecture** 120M-parameter bidirectional transformer: 12 layers, hidden 768, rotary positions, SwiGLU; MDLM (Sahoo et al., 2024), from scratch
- **data** 4.98B tokens of Telugu prose: Sangraha, IndicCorp v2, Wikipedia
- **compute** one 8 GB laptop GPU, 2 days 19 hours
- **generation** 512-token canvas, meaning given unmasked; lines decoded left to right within the line



# What pretraining bought us




*Training loss and validation NELBO over 76,000 optimizer steps of 65,536 tokens (from the project write-up; not in the paper). Validation NELBO 2.921 → 1.747; test NELBO 1.783 nats per token.*

## Trained on prose only

4.98B tokens of Telugu prose; no verse in pretraining. The deficits on the right, single-akṣara words and obscurity, are what poetry fine-tuning must fix next.

## Chandohasam 120M, free → constrained (Table 5)

**17.9% → 36.9%**

attested words: the constraint adds vocabulary instead of removing it

**183 → 757**

distinct attested words; the subword models reach 323–375

**0.006 → 0.035**

MAUVE against real verse of the same metre mix (real: 0.965)

**19.7% · 62.9%**

single-akṣara words (real 5.6%) and obscure poems (real 10.0%): no verse in pretraining

## Reading

One token per akṣara lets the constraint act on whole syllables, so masking removes options, not words. Form is free; what remains is to teach the model verse.

# Why poetry needs its own metrics, for sound and for sense

A poem can pass the controller and still be noise. Validity is binary; quality needs graded, deterministic measures, and meaning needs a metric of its own.

## śabdaṃ: deterministic, from the treatise

- **Source:** Kāvyālaṅkārasaṅgrahamu (16th c., Bhattumurthy) codifies anuprāsa, mādhurya, ojas, prasāda and the śabdālaṅkāras
- **caṃdomiti:** a graded distance to the nearest valid metre, with yati and prāsa edits, where the controller only says yes or no
- **śabdālaṅkāra identifiers:** adapted from the Sanskrit yamaka and anuprāsa identifiers; Telugu's alpha-syllabary allows it
- **Real verse as the yardstick:** only 2.7% strong anuprāsa, 10.0% obscure

## arthaṃ: the missing metric

- **Nothing existed:** Sanskrit systems fell back on human ratings (2.1–2.7 of 5); MAUVE is built for human-vs-AI detection and unvalidated on classical poetry
- **Semantic coherence:** to be modelled on expert human acceptability judgements; it cannot be reduced to formal constraints, so it is evaluated apart from the controller
- **Word resolution:** does each string resolve to a Telugu word through a macro-dictionary with sandhi and samāsa splitting?
- **arthālaṅkāras** are outside the scope: inherently semantic, not deterministic

# The poetry evaluation suite: four sound qualities

## anuprāsa · alliteration

$$z = (A - E) / \sqrt{V}$$

Repeated consonant sounds across different akṣaras within a line.

*A observed same-sound pairs; E expected under corpus unigram frequencies; V chance variance*

**Rubric:** Line levels at z = 1.645, 2.326, 3.090; poem score = mean line level

## mādhurya · euphony

$$M = I - P$$

Phonetic softness: the balance of soft and harsh akṣaras and the accumulation of harsh ones.

*I mean akṣara weight; P mean harsh-akṣara run load*

**Rubric:** harsh < −0.283 ≤ leaning harsh < −0.123 ≤ balanced < 0.049 ≤ leaning soft < 0.156 ≤ soft

## ojas · lexical density

$$O = (W + J) / 2$$

Word and consonant clustering: word-bound akṣaras and conjunct-initial akṣaras.

*W = 100 (1 − words / akṣaras); J = 100 (conjunct-initial akṣaras / akṣaras)*

**Rubric:** light < 39.3; leaning light; moderate; leaning dense; dense ≥ 47.5

## prasāda · clarity

$$H = -(1/N) \sum \log_2 P(a_t \mid a_{t-2}, a_{t-1})$$

Predictability of the akṣara sequence under a reference akṣara trigram model.

*bits per akṣara*

**Rubric:** clear < 5.12 ≤ leaning clear < 5.54 ≤ ordinary < 6.33 ≤ leaning obscure < 7.06 ≤ obscure

# caṃdomiti: a graded distance to the nearest valid metre

$$D = Dw + Dy + Dp$$
$$r = D / \max(n, N)$$
$$S = 1 - r / r_chance(m)$$

- **Dw, Dy, Dp** weight, yati and prāsa edits
- **n, N** akṣaras in the poem and in the nearest valid poem
- **r_chance(m)** mean edit rate of prose cut to the line lengths of metre m
- **S** 1 for a poem in metre, 0 for one no nearer to m than prose

## Rubric

exact (D = 0) · slip · partial · weak · unmetered (r at or above the metre-specific chance cut)

## How the edits are counted (Algorithm 4)

1. **Weight edits, per line**

   A dynamic programme over the slot's own automaton: reading an akṣara along an edge costs 0 if the edge's weight is one it may take, else 1 (substitution); skipping it costs 1 (deletion); an edge without reading costs 1 (insertion). A pāda-final laghu may read as U. The path gives the nearest legal line.

2. **Match lines to pādas**

   Lines are matched to the metre's pādas with the fewest edits; a line left over costs its length, a pāda left empty its shortest line.

3. **Yati edits**

   One for each yati seat whose akṣara is not in maitri with the vaḷi.

4. **Prāsa edits**

   The fewest pādas to change so that all the pādas rhyme. An akṣara a weight edit already rewrites is counted once.

# Metrics in design

## 1 Word resolution metric

- align every string with a compiled Telugu macro-dictionary
- sandhi and samāsa splitting, then word resolution
- measures word-level coherence across the poem
- doubles as a lexical gate: reject candidates with no valid segmentation

## 2 Semantic coherence metric

- modelled on expert human acceptability judgements of Telugu metrical poetry
- validated against the human ratings on a frozen test split
- cannot be reduced to formal constraints, so it is evaluated apart from the controller

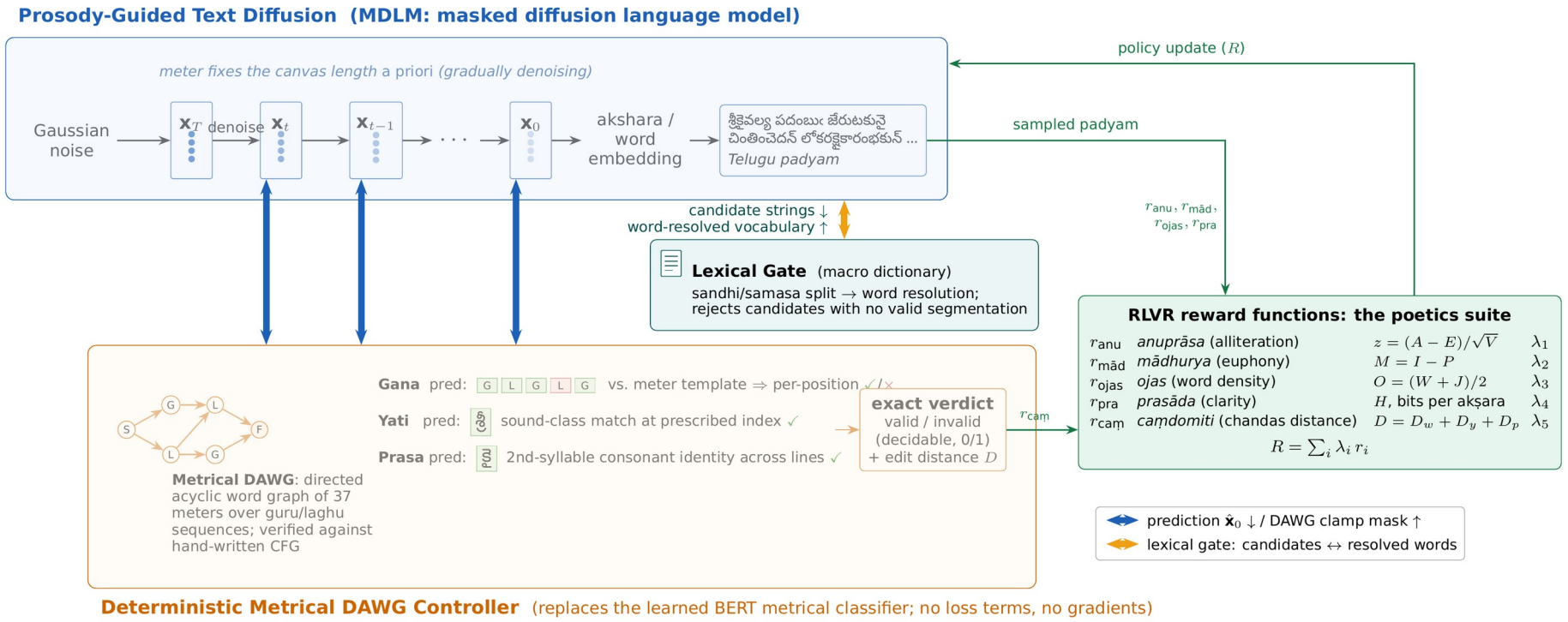
## 3 śabdālaṅkāra identifiers

- deterministic identifiers for figures of sound
- adapted from the Sanskrit yamaka and anuprāsa identifiers (Barbadikar and Kulkarni, 2023, 2024)
- arthālaṅkāras (figures of sense) remain outside the scope

## 4 Euphony index

- one score combining sound euphony, alliteration and word density
- its weights are learned against the human ratings
- all of the above feed the RLVR reward

# The Chandohasam framework: diffusion, controller, rewards




## Controller, not classifier

The DAWG replaces PoetryDiffusion's learned BERT metrical classifier: no loss terms, no gradients; an exact 0/1 verdict plus the edit distance D.

## Lexical gate

A macro-dictionary resolves words after sandhi and samāsa splitting and rejects candidates with no valid segmentation.

## RLVR: R = Σ λᵢ rᵢ

Poetics suite, śabdālaṅkāra checks and the coherence metric as verifiable rewards; the DAWG stays active, so RLVR optimises within the valid region.

# Progress so far

**12,673**

labelled poems in the benchmark, four sources, audited

**37**

metres in one labelled DAWG; 370,315 legal lines

**99.85%**

Bhāgavatamu poems identified with their labelled metre

**555 × 8**

poems per model and mode, four models, free and constrained

**5**

deterministic poetics metrics from Kāvyālaṅkārasaṅgrahamu

**120M**

parameters, our masked-diffusion model on 4.98B Telugu tokens

**Form is solved by construction.** What remains is meaning: evaluating and improving semantic coherence, extending the metrics, and modelling the coherence metric against human judgements.

# Next steps and timeline

1. **Word resolution metric**
   word-level coherence inside a poem, via the macro-dictionary
2. **Human evaluation**
   generated and reference poems; final rubric, frozen test split
3. **Semantic coherence metric**
   from the human acceptability judgements, validated against them; learn the euphony-index weights
4. **Poetry fine-tuning**
   the diffusion model on the curated corpus; meaning-to-poem setting and ablations
5. **RLVR**
   metrical constraints, evaluation suite and coherence metric as verifiable rewards; rejection-sampling and constrained baselines
6. **Final evaluation**
   all systems and the corpus: automatic metrics, MAUVE, human evaluation; automatic vs. human analysis
7. **Final report and release**
   controller, benchmark, evaluation suite, human ratings

**Final paper: 31 October 2026**
consolidate the results and release the controller, benchmark, evaluation suite and human ratings

# Demonstration: reading Pothana's opening verse

| pāda | మ UUU | స IIU | జ IUI | స IIU | త UUI | త UUI | గ U |
|---|---|---|---|---|---|---|---|
| 1 | శ్రీ కై వ | ల్య ప దం | బుఁ జే రు | ట కు నై | చిం తిం చె | దన్ లో క | ర |
| 2 | క్షై కా రం | భ కు భ | క్త పా ల | న క ళా | సం రం భ | కున్ దా న | వో |
| 3 | ద్రే క స్తం | భ కుఁ గే | ళి లో ల | వి ల స | ద్ధృ ర్జా ల | సం భూ త | నా |
| 4 | నా కం జా | త భ వాం | డ కుం భ | కు మ హా | నం దాం గ | నా డిం భ | కున్ |

**metre: śārdūlavikrīḍitamu ✓**

- ma sa ja sa ta ta ga, 19 akṣaras
- pāda-final laghu of line 1 read as guru

**yati 1 – 13 ✓ in every pāda**

- శ్రీ–చిం · క్షై–సం · ద్రే–ద్ధృ · నా–నం
- sound-class matches with the vaḷi

**prāsa on క ✓**

- second akṣara of each pāda: కై · కా · క · కం

```
$ python3 -m chandohasam --file poem.txt
metre: శార్దూలవిక్రీడితము (sardulavikriditamu)  ప్రాస ✓  యతి ✓  profile strict
$ python3 -m indic_meter_dawg prefix UUUI
kandamu/even, kandamu/odd, madhuragati_ragada/all, sardulavikriditamu/all      (28 → 14 → 7 → 4 → 3 → 1 as the prefix grows)
```

U U U I I U I U I I I U U U I U U I U

# Thank you

**Questions and discussion**

samvaran.rallabandi · mahesh.emani · radheshyam.s @research.iiit.ac.in

Chandohasam: a deterministic metrical controller, a curated benchmark, a poetry evaluation suite, and a diffusion model that writes one akṣara at a time.